William Herubel 2025-2026

Rapport de Stage

Stage du *23 mars au 5 juin*

Tuteur de stage : Fabrice Hoguin

Superviseur académique : Lin Yu WANG-RONG

Etablissement :

UVSQ université PARIS-SACLAY | IUT de Vélizy-Rambouillet
CAMPUS DE VÉLIZY-VILLACOUBLAY
CAMPUS DE RAMBOUILLET

Entreprise d'accueil :

UVSQ université PARIS-SACLAY | IUT de Vélizy-Rambouillet
CAMPUS DE VÉLIZY-VILLACOUBLAY
CAMPUS DE RAMBOUILLET

Remerciement **3**
Résumé (Version Française et Anglais) **4**
Introduction **5**
1. Etude économique de l'organisme d'accueil **6**
1.1. Présentation générale 6
1.2. Présentation de votre équipe de travail 6
2. Présentation du travail accompli durant le stage **7**
2.1. Panorama du stage 7
2.2. Présentation d'un projet 9
2.2.1. Présentation du projet 9
2.2.2. Version 1 du projet 11
2.2.3. Version 2 du projet 17
2.2.4. Version 3 : Sécurité et communication 20
2.2.5. Installation système 23
Conclusion : Bilan et Perspectives **25**
Glossaire **26**
Sitographie /Bibliographie **27**

Remerciement

En premier lieu, je tiens à remercier mon maître de stage, **M. Fabrice Hoguin.** Qui a supervisé mon stage et qui m’a permis d’apprendre de nouvelles méthodes à utiliser dans le langage Node.js.

Je le remercie de m’avoir proposé d'être installé dans une salle fermée au lieu d’une salle comme la E51 qui est ouverte dans les couloirs. Cela me permettait d’être dans une pièce calme ou avec un minimum de bruit me permettant de me concentrer pleinement sur mon travail.

Je suis vraiment reconnaissant envers ma famille pour leur présence pendant la partie de la recherche, car pendant que je cherchais de mon côté, ils cherchaient aussi grâce à leur contact. Je tiens également à exprimer ma gratitude pour leur aide précieuse, car ils m'ont prodigué des conseils et soutenu tout au long du stage.

Résumé (Version Française et Anglais)

Le stage a duré 41 jours ouvrés. Il a débuté le 23 mars et s'est terminé le 5 juin.
Avec une interruption du 17 avril au 4 mai durant laquelle il y avait des vacances scolaires pour me permettre d'être sur le campus en même temps que mon tuteur facilitant les échanges sur l'avancée du projet.

J'ai effectué mon stage au sein du campus de Vélizy, un Institut universitaire de technologie installé à Vélizy.

Durant ce stage, j'ai pu mettre en pratique mes connaissances en développement informatique, en planification de projet, en gestion de données. J'ai pu développer mes connaissances dans le langage Node.js et apprendre à utiliser de nouveaux modules et applications telles que Bcrypt et Mailjet.

À la fin de ce stage j'ai pu m'améliorer dans mes compétences en programmation en ayant participé à un projet professionnel durant tout son parcours que ce soit de sa création à sa mise en place sur le réseau interne. Cela m'a aussi permis de voir les différents problèmes qui peuvent avoir lieu que ce soit dans son déroulement ou dans sa mise en place et les conflits qui peuvent avoir lieu entre plusieurs infrastructures.

The internship lasted 41 working days. It began on 23 March and ended on 5 June.
There was a break from 17 April to 4 May, during the school holidays, to allow me to be on campus at the same time as my supervisor, making it easier to discuss the project's progress.

I undertook my internship at the Vélizy campus, a University Institute of Technology based in Vélizy.

During this internship, I was able to put my knowledge of software development, project planning and data management into practice. I was able to develop my knowledge of the Node.js language and learn to use new modules and applications such as Bcrypt and Mailjet.

By the end of this placement, I had improved my programming skills by participating in a professional project throughout its entire lifecycle, from its creation to its deployment on the internal network. This also allowed me to identify the various issues that can arise, whether during the project's execution or its implementation, as well as the conflicts that can occur between different infrastructures.

Introduction

Durant la période allant du 23 mars au 5 juin, j’ai effectué un stage au sein du campus de Vélizy. Ce stage avait pour objectif de me permettre d'approfondir mes connaissances dans le domaine du développement web, notamment en Node.js, et de découvrir le fonctionnement d’un environnement professionnel informatique.

Le campus de Vélizy, situé à Vélizy-Villacoublay, a été créé en 1991 et est spécialisé dans les formations liées aux technologies industrielles et tertiaires. L’établissement est une composante de l’université de Versailles-Saint-Quentin-en-Yvelines, une université française située dans les Yvelines et créée en 1991. Au cours de ce stage, j’ai été accompagné par mon maître de stage Fabrice Hoguin qui est au poste d'enseignant et avec qui j’ai pu me former dans d’excellentes conditions.

Lors de ce stage, j'ai principalement travaillé sur la création de sites et le développement de composants informatiques. Cette expérience m'a offert l'opportunité d'approfondir mon niveau en Node.js et de découvrir de nouvelles bibliothèques ainsi que différents systèmes et outils de développement. Cela m’a permis d'enrichir mes connaissances dans le développement d'applications et dans la création de sites web.

L’élaboration de ce rapport a pour principal objectif de rassembler les différentes connaissances acquises durant ce stage, les différentes missions réalisées ainsi que les enseignements tirés de cette expérience professionnelle.

Afin de présenter de manière claire et détaillée les deux mois passés au sein du stage, je commencerai par présenter l’établissement et son environnement. Puis, en deuxième partie, j’aborderai les différentes tâches et missions que j'ai accomplies lors de cette formation, ainsi que les compétences et les connaissances acquises au cours de ce stage.

1. *Etude économique de l'organisme d'accueil*

1.1. Présentation générale

J'ai réalisé mon stage à l'Institut universitaire de technologie de Vélizy-Rambouillet, situé dans la ville de Vélizy-Villacoublay en France. Cet établissement fait partie de l'Université de Versailles – Saint-Quentin-en-Yvelines (UVSQ) implantée dans le département des Yvelines.

L'IUT de Vélizy-Rambouillet a été créé en 1991 à Vélizy lors du développement de l'UVSQ. Un second site a ensuite été ouvert à Rambouillet en 1994 avant d'être agrandi en 1999. Aujourd'hui, l'établissement propose plusieurs formations universitaires professionnalisantes, comprenant notamment 3 licences professionnelles et 7 Bachelors Universitaires de Technologie (BUT).

Le site de Vélizy accueille principalement les formations :
- BUT Génie électrique et informatique industrielle;
- BUT Informatique - option génie informatique;
- BUT Réseaux et Télécommunications;
- BUT Services et réseaux de communication;

Le site de Rambouillet propose quant à lui les formations :
- BUT Génie chimique - génie des procédés - option Bioprocédés;
- BUT Gestion administrative et commerciale;
- BUT Techniques de commercialisation;

L'IUT de Vélizy-Rambouillet compte aujourd'hui environ 70 enseignants et enseignants-chercheurs, plus de 160 intervenants extérieurs ainsi que 44 personnels administratifs. Chaque année, l'établissement accueille plus de 1 200 étudiants répartis dans les différentes filières de formation.

En 2022, l'établissement a été classé 22ème de France sur l'ensemble des critères pris en compte, 23ème sur les critères d'attractivité et de sélectivité, et 22ème sur le classement complet qui fut révélé par Tholis. Un classement qui sera stabilisé car pour le classement de 2026, avec une 22ème place au classement national des IUT.

1.2. Présentation de votre équipe de travail

Pour ce projet, j'ai travaillé de manière autonome. Le développement a donc été réalisé individuellement, ce qui m'a permis de prendre l'ensemble des décisions techniques ainsi que de choisir les outils et technologies à mettre en œuvre.

Le seul intervenant extérieur était mon tuteur de stage, M. Fabrice Hoguin, qui assurait le suivi régulier de l'avancement du projet. Il intervenait pour valider les étapes réalisées, proposer des améliorations et suggérer l'ajout de nouvelles fonctionnalités à l'application.

2. *Présentation du travail accompli durant le stage*

2.1. Panorama du stage

L'objectif de ce stage était de mener un projet de développement dans son intégralité, en passant par les différentes étapes de conception jusqu'à la phase de développement. Le sujet du projet était libre dans sa conception, sous réserve de respecter certains critères définis en amont.

Le projet consistait en la réalisation d'une application web destinée à un service de reprographie. Cette application devait être développée avec le framework Express côté serveur, ainsi qu'un framework côté client.

L'objectif de l'application était de mettre en place un tableau de bord permettant la gestion des demandes de reprographie. Plusieurs fonctionnalités ont été définies dès le début du projet, notamment :

- l'authentification des utilisateurs avec des fonctionnalités de connexion et déconnexion,
- le dépôt de fichiers tels que (PDF, DOC, DOCX),
- la mise en place d'un système de notifications indiquant la prise en charge des fichiers par le service,
- l'envoi de notifications lorsque les travaux sont terminés,
- la gestion des notifications en cas d'erreur lors du traitement des demandes

Au cours du stage, le projet a évolué afin d'ajouter de nouvelles fonctionnalités et de protection de sécurité parmi lesquelles ont peut retrouver le système de protection CSRF, ainsi que l'utilisation du service Mailjet pour permettre l'envoi automatique de courriels.

Outil Informatique :

Lors de la réalisation de ce projet, plusieurs outils et logiciels ont été utilisés afin de permettre le développement et la gestion de l'application web.

- **Node.js** : principal environnement utilisé pour le développement de l'application. Node.js est un environnement d'exécution JavaScript open source et multiplateforme permettant de développer des serveurs, des applications web ainsi que des outils en ligne de commande et des scripts.
- **MongoDB** : système de base de données utilisé pour stocker les informations de l'application. MongoDB est un système de gestion de base de données orienté documents appartenant à la catégorie des systèmes NoSQL.
- **MongoDB Compass** : outil graphique permettant d'interagir avec la base de données MongoDB. Il a notamment servi à visualiser, modifier et gérer les données stockées dans la base du projet.
- **Visual Studio Code** : outil permettant de développer du code. Visual Studio Code est un éditeur de code extensible développé par Microsoft pour Windows, Linux et

macOS. Il permet d'utiliser une variété de langages de programmation, tels que Java, JavaScript, Go, Node.js, C++.

- **Git/GitHub :** outils utilisés pour le suivi et la sauvegarde du projet. Git permet la gestion de versions du code source, tandis que GitHub a servi à héberger le projet et à conserver un historique des modifications effectuées durant le développement.
- **Express.js :** Framework utilisé pour la partie serveur du projet. Express.js est un framework permettant la construction d'applications web basées en Node.js.
- **Express-session** : middleware utilisé pour la gestion des sessions utilisateur et des cookies. Express-session est un middleware Express.js qui permet de gérer les sessions utilisateur, permettant de stocker les données spécifiques à l'utilisateur sur le serveur.
- **Express-fileupload** : middleware utilisé pour la gestion des fichiers. Express-fileupload est un middleware Express.js qui simplifie le téléversement de fichiers dans une application Node.js
- **Mongoose** : outil utilisé pour permettre la communication entre l'application Node.js et MongoDB. Mongoose est un outil de modélisation d'objets MongoDB conçu pour fonctionner dans un environnement asynchrone.
- **Bcrypt** : outil utilisé pour sécuriser les mots de passe des utilisateurs grâce au chiffrement des données sensibles avant leur stockage dans la base de données. Bcrypt est une fonction de dérivation de clé basée sur l'algorithme de chiffrement Blowfish. En plus de l'utilisation d'un sel pour se protéger des attaques par table arc-en-ciel (rainbow table), bcrypt est une fonction adaptative, c'est-à-dire que l'on peut augmenter le nombre d'itérations pour la rendre plus lente.
- **Csurf** : outil permettant de sécuriser l'application contre les attaques CSRF. Csurf est un middleware pour Node.js qui empêche les attaques de type Cross-Site Request Forgery (CSRF) sur une application.
- **Node-mailjet** : bibliothèque utilisée pour exploiter l'API Mailjet afin d'envoyer automatiquement des e-mails depuis l'application. Mailjet est un service cloud spécialisé dans l'envoi et le suivi d'e-mails.

Formations et autoformations :

Pour réaliser ce projet, j'ai dû utiliser diverses bibliothèques, logiciels ou outils, si pour certains je les connaissais déjà tels que Node.js et Express que j'ai appris à utiliser lors des projets universitaires. Pour d'autres je devais réaliser diverses recherches et d'autoformation grâce aux documentations et code publié sur internet.

- Pour **MongoDB**, ce fut une recherche réduite car son fonctionnement est assez similaire à d'autres langages de base de données, ce qui changeait était les noms utilisés, le fonctionnement et les commandes à utiliser. J'ai donc suivi la documentation officielle de MongoDB et de Mongoose qui donne des exemples de code que j'ai ensuite modifiés.
- Pour **Csurf**, ce fut une recherche plus longue, il me fallait comprendre ce qu'était csurf mais aussi à quoi il servait et comment l'utiliser dans l'application. Pour cela je lisais les différents documents disponibles sur internet et je regardais le GitHub officiel du projet qui ont un README informant de l'utilisation.
- Pour **Mailjet**, ce fut une recherche moins poussée, contrairement aux autres composants et bibliothèques, Mailjet possède un site et un service avec lequel ils proposent un tutoriel pour prendre en main leur API et comment l'utiliser sur

différents langages. Ils donnent aussi des morceaux de code déjà réalisés pour faciliter la mise en place.

2.2. Présentation d'un projet

2.2.1. Présentation du projet

Le but du projet est de développer une application web permettant de centraliser les demandes de reprographie et de faciliter la visualisation de l'avancement des demandes des utilisateurs.

Actuellement, le système de reprographie repose sur l'envoi de courriels à la personne chargée de la reprographie. Ces messages contiennent une description de la demande ainsi que le fichier à imprimer. Cependant, ce fonctionnement présente plusieurs limites. Il ne permet pas de suivre l'avancée de la demande avant d'aller récupérer les documents. De plus, un même document peut être envoyé plusieurs fois par erreur,ce qui entraîne des doublons.

Afin de répondre à ces problématiques, une première version de l'application m'a été demandée. Celle-ci devait constituer une base évolutive, amenée à être améliorée au cours du stage. Cette première version devait inclure les fonctionnalités suivantes

- La possibilité de se connecter et de se déconnecter d'un compte utilisateur ;
- La création de demandes (appelées "tickets") ;
- La visualisation des demandes de l'utilisateur ;
- La prise en charge des tickets par le service de reprographie ;
- La fermeture (ou validation) des tickets une fois la demande traitée ;

Avant de détailler la conception de l'application, il est nécessaire de présenter le contexte fonctionnel ainsi que les différents acteurs intervenant dans le système de reprographie.

La reprographie désigne l'ensemble des procédés permettant l'impression et la reproduction d'un document original, sur un support papier.

Dans le cadre de ce projet, trois types d'acteurs ont été identifiés :

1. **Le Visiteur**

Le visiteur correspond à un utilisateur non authentifié, il peut accéder à la page d'accueil de l'application et dispose de la possibilité de s'inscrire en créant un nouveau compte ou de se connecter afin d'accéder aux fonctionnalités de l'application. L'accès aux fonctionnalités est donc conditionné par l'authentification.

2. **L'utilisateur**

L'utilisateur est un acteur authentifié. Une fois connecté, il peut créer des demandes de reprographie sous forme de tickets via un formulaire. Il peut également consulter l'historique et le statut de ses propres demandes, ainsi que se déconnecter de l'application.

3. Le Reprographe

Le reprographe est responsable du traitement des demandes. Il est également un utilisateur authentifié. Il dispose d'un accès à l'ensemble des tickets créés par les utilisateurs. Ses principales actions sont :

- consulter les demandes
- modifier l'état d'un ticket
- télécharger les documents associés
- clôturer les tickets une fois le traitement terminé

Chacune des demandes réalisés par un utilisateur est représentée sous la forme d'un ticket contenant les informations suivantes :

- Un identifiant unique
- Un nom de document
- Le type du document
- Le nombre de pages du document
- Le nombre d'exemplaires à imprimer
- Une option indiquant si l'impression doit être réalisée en recto ou en recto-verso
- Une option indiquant si les documents doivent être agrafés
- La date de création du ticket
- La date à laquelle le document doit être disponible
- L'état du ticket
- L'utilisateur qui a crée la demande
- Le reprographe ayant pris en charge la demande

Chaque ticket possède trois états possibles :

- Ouvert
- En cours de traitement
- Complété

Le cycle de vie d'un ticket suit la logique suivante :
Création par l'utilisateur -> Prise en charge par le reprographe -> clôture après traitement.

Chaque utilisateur dispose d'un compte contenant les informations suivantes :

- Un identifiant
- Un nom
- Un prénom
- Une adresse mail unique
- Un mot de passe chiffré
- Un département
- Un rôle associé.

Les mots de passe sont stockés de manière chiffrée afin de garantir la sécurité du système.

Les utilisateurs sont associés à un département parmi les suivants : INFO, GEII, MMI, RT. Cette liste peut évoluer en fonction des besoins du projet.

Afin de répondre aux besoins identifiés, l'application a été organisée autour de plusieurs pages web dédiées aux différents acteurs du système.

Les visiteurs peuvent accéder aux pages d'inscription et de connexion afin de créer un compte ou de s'authentifier. Une fois connecté, l'utilisateur dispose d'un espace lui permettant de créer des demandes de reprographie et de consulter l'historique de ses tickets ainsi que leur état d'avancement.

Le reprographe dispose quant à lui d'une interface dédiée regroupant l'ensemble des demandes afin d'en assurer le traitement. Cette interface lui permet notamment de consulter les tickets, de télécharger les documents associés et de mettre à jour leur état.

Cette organisation a servi de base à la conception de l'application et à la mise en place des différentes fonctionnalités développées au cours du stage.

2.2.2. Version 1 du projet

L'objectif de cette version est de réaliser les fonctionnalités rédigées dans la partie précédente et mettre les pages web.

Pour cette première version, l'application a été structurée autour de six pages principales. Afin d'assurer une expérience utilisateur cohérente, certains composants ont été intégrés à l'ensemble des pages. Parmi eux, une barre de navigation facilite l'accès aux différentes fonctionnalités de l'application, tandis qu'un pied de page regroupe les informations complémentaires et participe à l'harmonisation de l'interface.

Fig 1 : Exemple de barre de navigation

La première page développée fut celle de l'accueil. Elle a pour objectif d'être le cœur de navigation de l'application. Première page visitée par l'utilisateur, elle doit assurer une navigation fluide entre les différentes pages et lui permettre d'accéder rapidement aux fonctionnalités principales.

Pour ce faire, cette page ne doit pas contenir d'informations parasites et doit mettre en évidence les éléments essentiels de navigation, afin de permettre à un utilisateur non expérimenté de comprendre facilement le fonctionnement de l'application et comment naviguer.

Dans cette première version, elle se limite à une barre de navigation ainsi qu'un pied de page. Elle pourra toutefois évoluer ultérieurement afin d'intégrer un contenu explicatif destiné aux nouveaux utilisateurs.

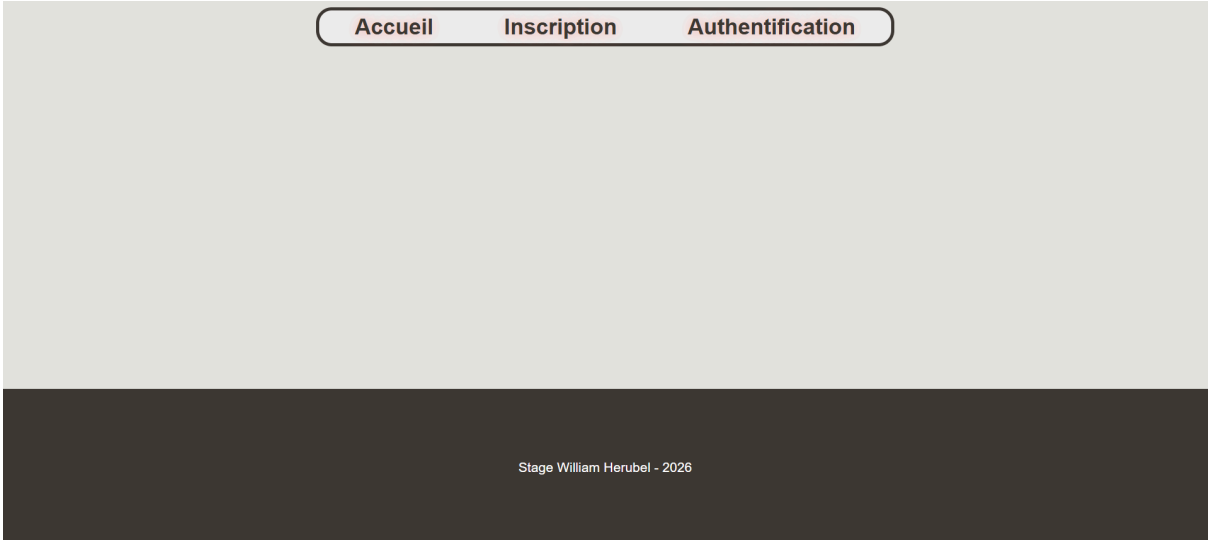


Fig 2 : Page Accueil Version 1

La deuxième page développée fut celle de l'inscription. Elle a pour objectif de permettre à un utilisateur de créer un compte nécessaire à l'utilisation de l'application. Première étape du parcours utilisateur, elle doit garantir une saisie claire et fluide des informations afin d'éviter les erreurs et de faciliter la compréhension des champs à compléter.

Pour ce faire, la page est construite sous la forme d'un formulaire regroupant l'ensemble des informations nécessaires à la création d'un compte. Les champs sont organisés verticalement afin de favoriser la lisibilité et de guider l'utilisateur dans la saisie, de manière similaire à une lecture linéaire.

En bas du formulaire, deux boutons sont proposés : un bouton permettant de l'inscrire avec les informations saisies, et un second permettant de rediriger l'utilisateur vers la page de connexion en cas d'un changement d'avis.

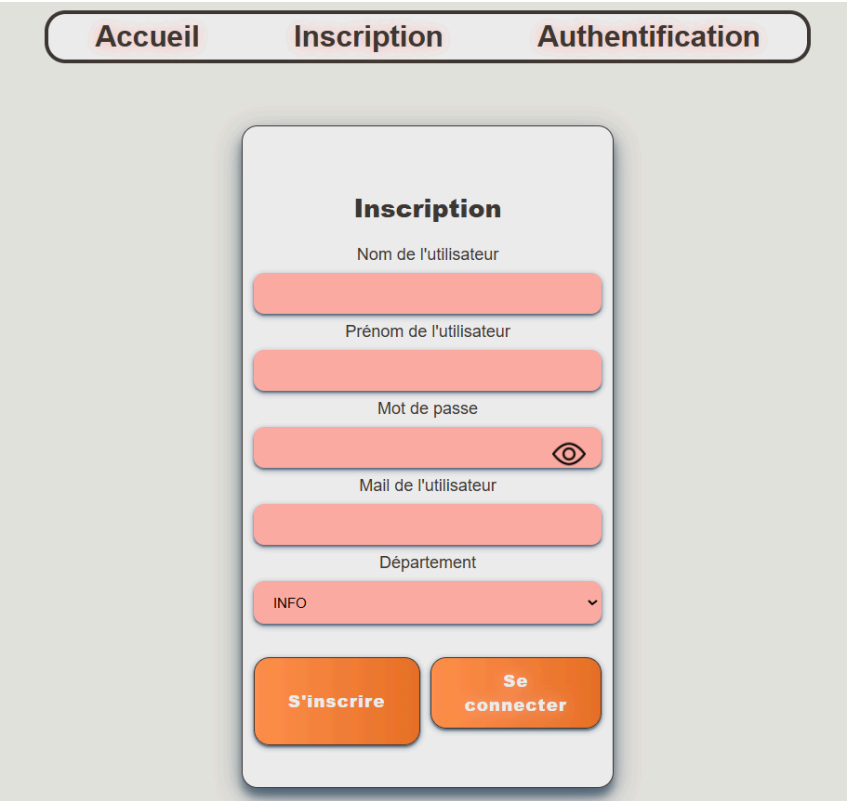


Fig 3 : Page Inscription Version 1

La troisième page développée fut celle de la connexion. Elle a pour objectif de permettre à un utilisateur de s'authentifier à son compte. Première étape du parcours utilisateur authentifié, elle doit garantir une saisie claire et fluide des informations afin d'assurer la vérification de l'identité de l'utilisateur.

Pour ce faire, la page est construite sous la forme d'un formulaire regroupant l'ensemble des informations nécessaires à la connexion. Les champs sont organisés verticalement afin de favoriser la lisibilité et de guider l'utilisateur dans la saisie, selon une logique de lecture linéaire.

En bas du formulaire, deux boutons sont proposés : un bouton permettant de valider la connexion avec les informations saisies, et un second redirigeant l'utilisateur vers la page d'inscription en cas de besoin.

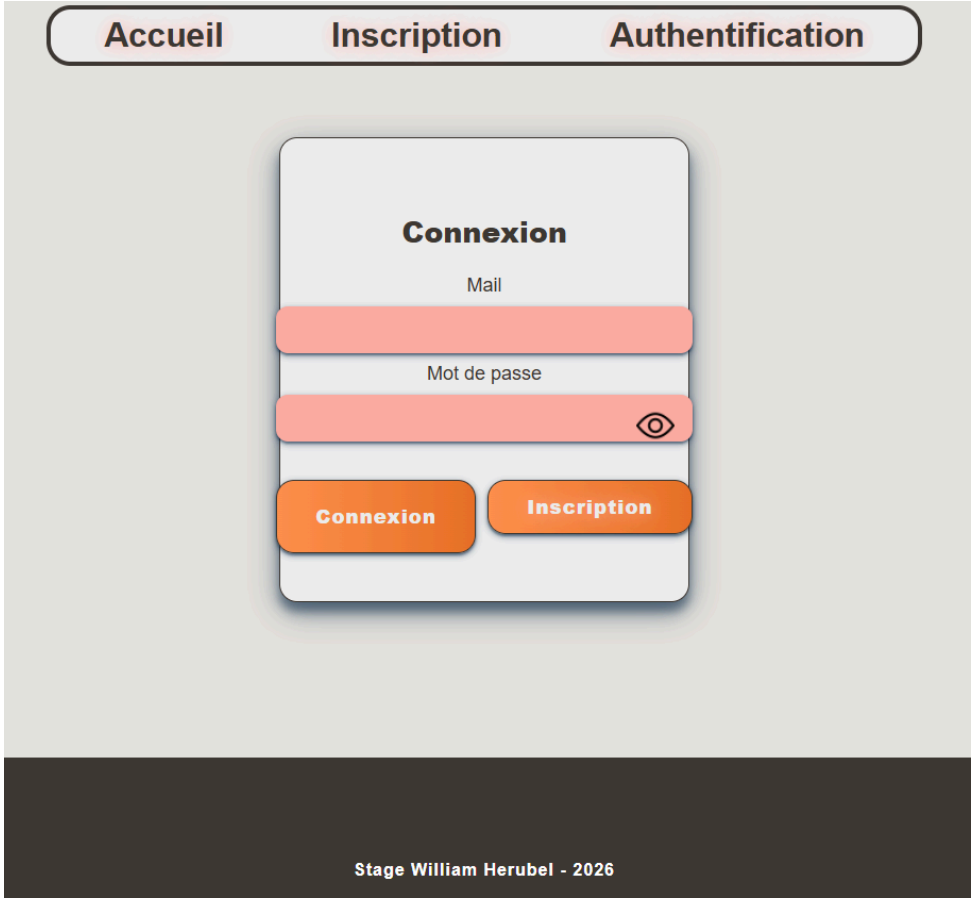


Fig 4 : Page Connexion Version 1

La quatrième page développée fut celle du formulaire de création de tickets. Elle a pour objectif de permettre à un utilisateur de rédiger des demandes d'impression de documents. Fonctionnalité principale du compte utilisateur, elle doit garantir la saisie de l'ensemble des informations nécessaires au traitement de la demande.

Pour ce faire, la page est construite sous la forme d'un formulaire permettant de regrouper toutes les informations relatives au document et à son impression. Il est divisé en trois parties. La première concerne les informations générales du document, telles que son nom et son type. La deuxième porte sur les contraintes de délai notamment la date de retour souhaitée. La troisième regroupe les paramètres d'impression, tels que le nombre de pages et le nombre d'exemplaires.

Les champs sont organisés verticalement afin de favoriser la lisibilité et de guider l'utilisateur dans la saisie progressive, selon une logique de lecture linéaire.

En bas du formulaire, un bouton est proposé permettant de valider le formulaire avec les informations saisies qui redirigera ensuite vers le tableau de bord.

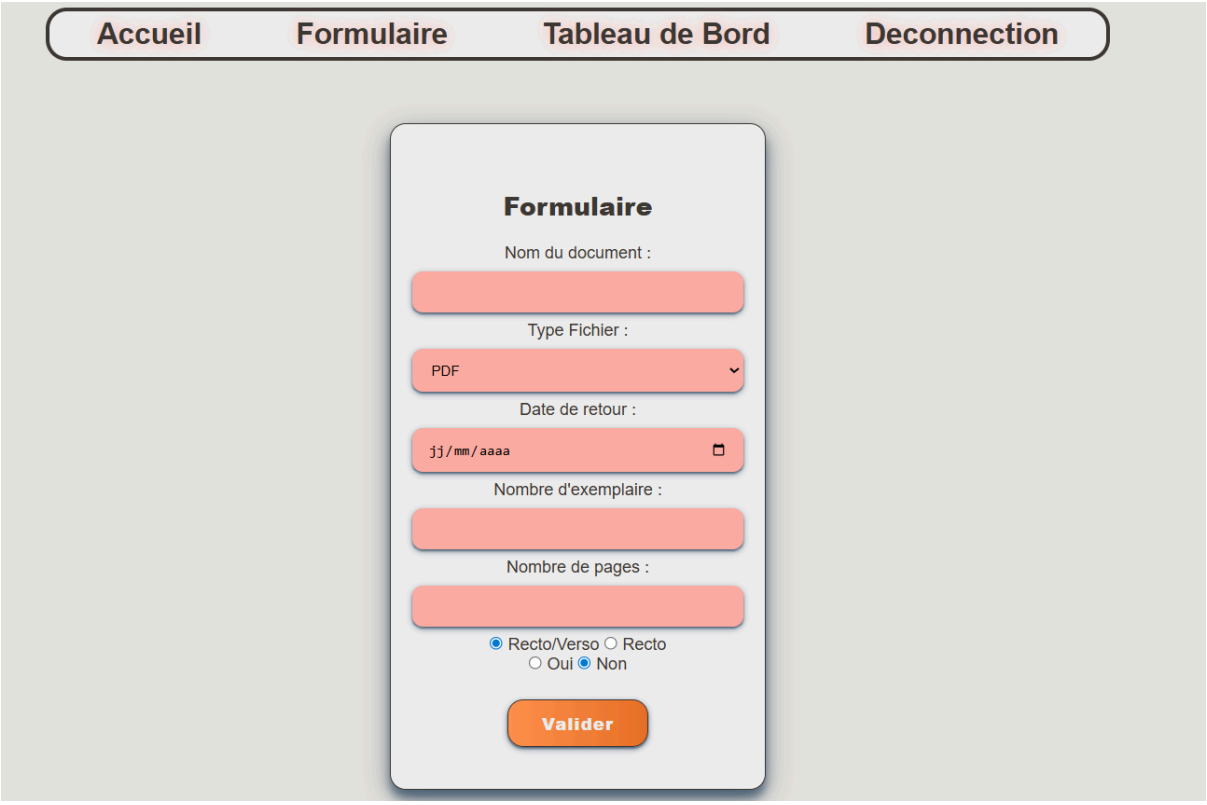
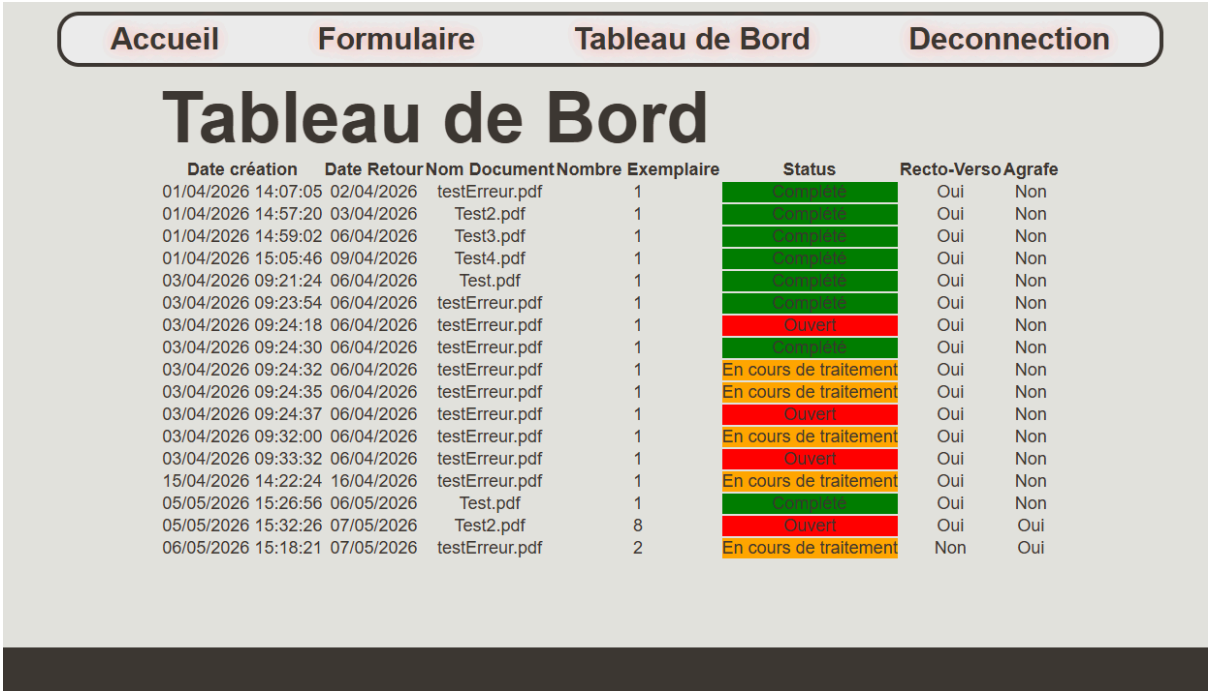


Fig 5 : Page Formulaire Version 1

La cinquième page développée fut celle du tableau de bord. Elle a pour objectif de permettre à un utilisateur de visualiser l'ensemble des tickets qu'il a créés. Page principale de l'utilisateur, elle doit lui permettre d'accéder rapidement aux informations essentielles sans avoir à les rechercher parmi un grand volume de données.

Pour ce faire, la page contient un tableau regroupant les tickets créés par l'utilisateur. Celui-ci présente un nombre réduit d'informations afin de mettre en avant les éléments les plus pertinents. Par exemple, le type de document n'est pas affiché dans une colonne distincte, mais elle est associée au nom du document afin de faciliter la compréhension globale de la demande.

La colonne « Statut » est l’information la plus importante du tableau. Elle est mise en avant grâce à un système de couleurs permettant une lecture rapide de l’état d’avancement des tickets. Ainsi, lorsqu'un ticket est simplement créé et n’a pas encore été pris en charge par un reprographe, son statut est affiché en rouge afin d’indiquer l’absence de traitement. Lorsqu'un statut est affiché en orange, il indique alors que le ticket est pris en charge par un reprographe mais il n’est pas encore possible de le récupérer. Enfin lorsque le statut est affiché en vert il indique que reprographe ayant récupéré le ticket a finalisé l’impression et qu’il est alors possible de récupérer le document.



Date création	Date Retour	Nom Document	Nombre Exemplaire	Status	Recto-Verso	Agrafe
01/04/2026 14:07:05	02/04/2026	testErreur.pdf	1	Complété	Oui	Non
01/04/2026 14:57:20	03/04/2026	Test2.pdf	1	Complété	Oui	Non
01/04/2026 14:59:02	06/04/2026	Test3.pdf	1	Complété	Oui	Non
01/04/2026 15:05:46	09/04/2026	Test4.pdf	1	Complété	Oui	Non
03/04/2026 09:21:24	06/04/2026	Test.pdf	1	Complété	Oui	Non
03/04/2026 09:23:54	06/04/2026	testErreur.pdf	1	Complété	Oui	Non
03/04/2026 09:24:18	06/04/2026	testErreur.pdf	1	Ouvert	Oui	Non
03/04/2026 09:24:30	06/04/2026	testErreur.pdf	1	Complété	Oui	Non
03/04/2026 09:24:32	06/04/2026	testErreur.pdf	1	En cours de traitement	Oui	Non
03/04/2026 09:24:35	06/04/2026	testErreur.pdf	1	En cours de traitement	Oui	Non
03/04/2026 09:24:37	06/04/2026	testErreur.pdf	1	Ouvert	Oui	Non
03/04/2026 09:32:00	06/04/2026	testErreur.pdf	1	En cours de traitement	Oui	Non
03/04/2026 09:33:32	06/04/2026	testErreur.pdf	1	Ouvert	Oui	Non
15/04/2026 14:22:24	16/04/2026	testErreur.pdf	1	En cours de traitement	Oui	Non
05/05/2026 15:26:56	06/05/2026	Test.pdf	1	Complété	Oui	Non
05/05/2026 15:32:26	07/05/2026	Test2.pdf	8	Ouvert	Oui	Oui
06/05/2026 15:18:21	07/05/2026	testErreur.pdf	2	En cours de traitement	Non	Oui

Fig 6 : Page Tableau de bord Version 1

La sixième page développée fut celle du reprographe. Elle a pour objectif de permettre à un reprographe de visualiser l’ensemble des tickets générés par les utilisateurs et de modifier leurs états. Page principale du reprographe, elle lui permet d’identifier les tickets disponibles ainsi que les tickets en cours de traitement. Elle donne également accès aux informations nécessaires à leur prise en charge, telles que les paramètres d’impression et la date limite associée à chaque demande.

Pour ce faire, la page est organisée en deux parties. La partie supérieure regroupe les tickets ouverts, permettant au reprographe de les consulter et de les prendre en charge. La partie inférieure affiche les tickets déjà pris en charge, offrant la possibilité de les finaliser et de télécharger les documents à imprimer.

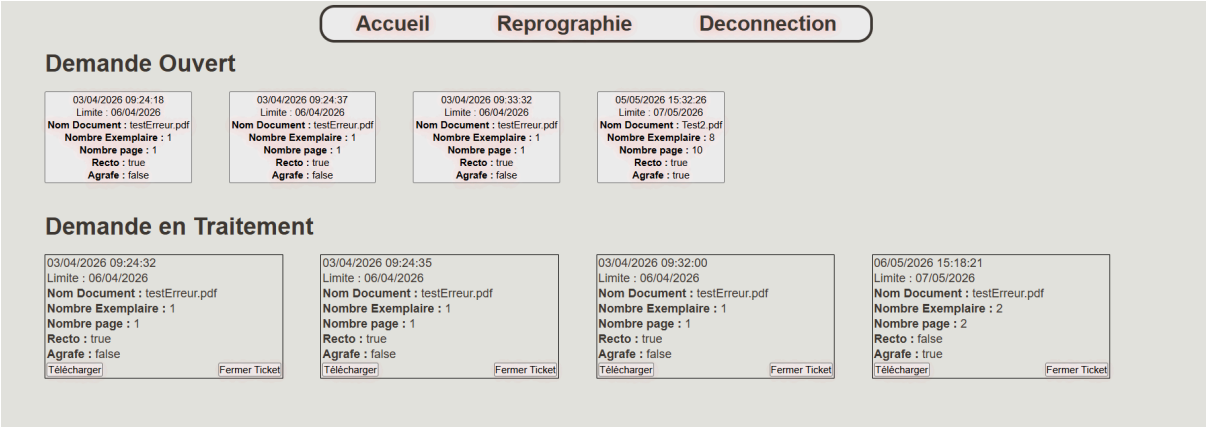


Fig 7 : Page Reprographe Version 1

Ainsi s'achève le développement de la première version de l'application, comprenant six pages permettant à un utilisateur de se connecter, de s'inscrire, de créer des tickets et de les consulter, l'application permet également de prendre en charge les demandes et d'en assurer le suivi.

Cependant, si la gestion des tickets est fonctionnelle, le traitement des fichiers n'est pas encore implémenté. Il est actuellement impossible de déposer des documents, ce qui empêche leur téléchargement par le reprographe. Cette limite restreint l'utilisation de l'application et la rend incomplète au regard du cahier des charges défini en section 2.2.1.

Par ailleurs, la sécurité de l'application reste limitée. À ce stade, la seule protection mise en place concerne la validation des champs côté interface lors de l'inscription, empêchant l'envoi du formulaire lorsque certaines conditions ne sont pas respectées.

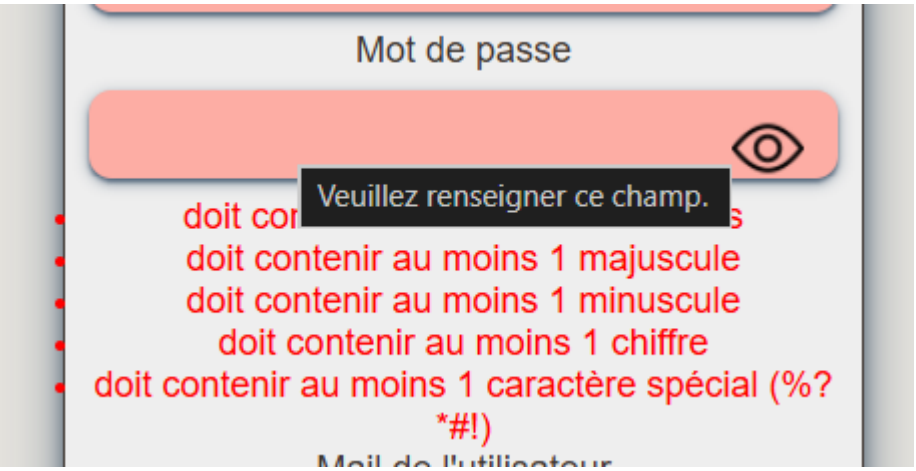


Fig 8 : Critère d'inscription

2.2.3. Version 2 du projet

L’objectif de cette version est d’ajouter les fonctionnalités encore manquantes de l’application, notamment la gestion des fichiers, d’intégrer un nouveau rôle utilisateur et de faire évoluer certaines fonctionnalités afin de les rendre plus dynamiques, comme la gestion de la liste des départements.

Pour cette version un nouveau rôle à était ajouté : l'administrateur. Son rôle est de configurer les paramètres de l'application et d’assurer son administration. Il dispose ainsi de droit lui permettant de manipuler la base de données afin d’ajouter , modifier ou supprimer certains éléments. Il peut également créer de nouveaux comptes reprographes afin de gérer les accès au service de reprographie.

L'ajout de ce rôle permet de séparer les tâches d'administration des tâches de reprographie, tout en facilitant la maintenance et l'évolution de l'application.

Pour mettre en place ce nouveau rôle, une nouvelle page a été développée. Elle a pour objectif de centraliser les différentes actions d’administration de l’application. En tant que page principale de l’administrateur, elle lui permet de créer de nouveaux comptes reprographes ainsi que de gérer certaines données de la base de données, notamment les listes utilisées par l'application comme celle des départements.

Pour ce faire, la page est divisée en deux parties distinctes.

La première partie prend la forme d’un formulaire, similaire à celui utilisé pour l’inscription des utilisateurs. Elle permet de saisir les informations nécessaires à la création d’un nouveau compte. La principale différence réside dans le choix du rôle associé au compte, qui peut être défini comme « Reprographe ».

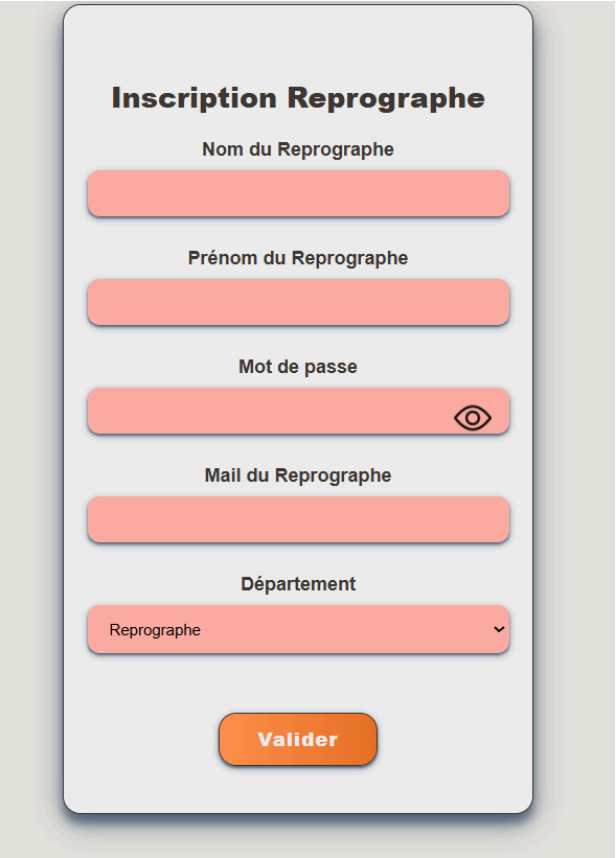


Fig 9 : Page Administrateur – Création d'un compte reprographe

La seconde partie est dédiée à la gestion des départements. Elle permet de visualiser la liste des départements disponibles ainsi que d'effectuer différentes opérations sur celle-ci. Trois actions sont proposées : l'ajout d'un nouveau département, la modification d'un département existant et la suppression d'un département.

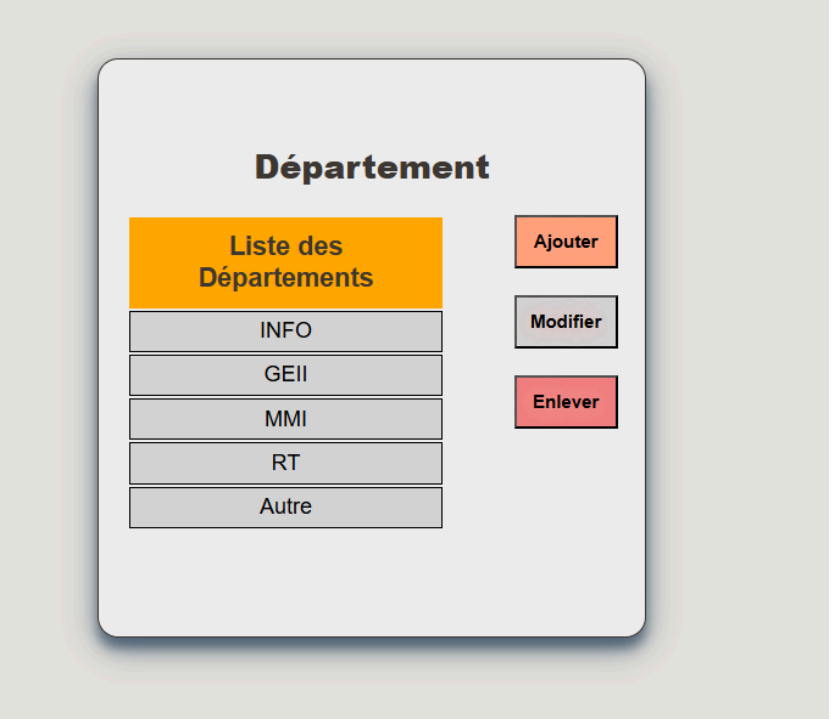


Fig 10 : Page Administrateur – Gestion des départements

Cette fonctionnalité permet de rendre l'application plus flexible en évitant de devoir modifier directement le code source pour ajouter ou modifier un département ou créer un nouveau compte reprographe.

L'application utilise une base de données MongoDB afin de stocker les informations relatives aux utilisateurs et aux tickets. Cependant, cette base de données peut également être utilisée pour rendre certaines données de l'application dynamiques. C'est notamment le cas de la liste des départements, qui était jusqu'à présent définie directement dans le code de l'application.

L'ajout de la page administrateur a permis de modifier cette liste sans intervention sur le code source. Il était donc nécessaire de trouver un moyen de rendre ces modifications accessibles à l'ensemble des pages de l'application.

Pour répondre à ce besoin, une collection spécifique a été créée dans MongoDB afin de stocker les différents départements. À chaque chargement d'une page nécessitant cette information, l'application interroge la base de données et récupère la liste des départements. Les données sont ensuite enregistrées dans une variable locale transmise à la vue afin de générer dynamiquement les éléments de l'interface.

Cette solution permet d'ajouter, de modifier ou de supprimer un département depuis l'interface d'administration sans avoir à modifier le code de l'application ni à effectuer un nouveau déploiement.

L'une des fonctionnalités ajoutées lors de cette version est la gestion des fichiers. Elle permet à un utilisateur de déposer un document lors de la création d'un ticket et au reprographe de le télécharger afin de procéder à son impression.

Pour mettre en place cette fonctionnalité, le middleware Express-fileupload a été utilisé. Ce module simplifie la gestion des fichiers dans une application Express en fournissant des méthodes permettant de récupérer les fichiers envoyés par l'utilisateur et de les enregistrer sur le serveur.

La première étape a consisté à modifier la page de création de ticket afin d'ajouter un champ permettant le dépôt d'un fichier. L'utilisateur peut ainsi sélectionner le document à imprimer directement lors de la saisie de sa demande.

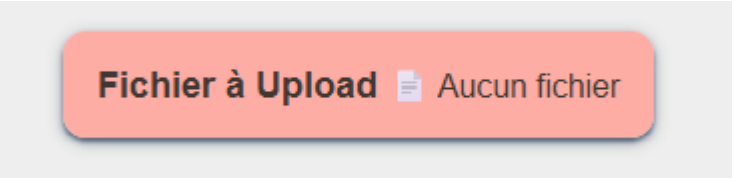


Fig 11 : Page Formulaire - Fonctionnalité upload

Lors de la validation du formulaire, le fichier transmis est récupéré par le serveur puis enregistré dans un dossier dédié de l'application. Pour cela, la méthode `sampleFile.mv(uploadPath)` est utilisée. Cette méthode permet de déplacer le fichier reçu vers l'emplacement défini dans la variable `uploadPath`, correspondant au chemin de stockage du document sur le serveur.

Une fois le fichier enregistré, son emplacement est associé au ticket créé dans la base de données. Cette information pourra ensuite être utilisée par le reprographe afin de télécharger le document depuis l'interface de gestion des tickets.

Du côté du reprographe, le bouton de téléchargement du document, présent sur les tickets pris en charge, a été rendu fonctionnel. Cette fonctionnalité permet au reprographe de récupérer directement le fichier associé à une demande afin de procéder à son impression.

Pour cela, la méthode `res.download(path, name)` d'Express a été utilisée. Cette méthode permet d'envoyer un fichier au navigateur de l'utilisateur sous la forme d'un téléchargement. Le paramètre `path` correspond au chemin de stockage du document sur le serveur, tandis que le paramètre `name` définit le nom qui sera attribué au fichier lors du téléchargement.

Ainsi, lorsqu'un reprographe sélectionne l'option de téléchargement d'un ticket, l'application récupère le chemin du fichier enregistré dans la base de données puis transmet le document au navigateur, permettant son téléchargement immédiat.

Ainsi s'achève le développement de la deuxième version de l'application. Cette évolution a permis l'ajout d'un nouveau rôle utilisateur et de la page d'administration associée, la mise en place d'un système dynamique pour la gestion des listes de données, l'implémentation de fonctionnalités majeures telles que la gestion des fichiers ainsi que l'ajout de messages d'erreur et de confirmation destinés à améliorer l'expérience utilisateur.

Cette version permet désormais de répondre à l'ensemble des fonctionnalités définies dans le cahier des charges initial et aboutit à une application pleinement fonctionnelle sur le plan métier.

Cependant, bien que l'application soit désormais opérationnelle, elle reste encore vulnérable à certaines attaques informatiques. Les mécanismes de sécurité mis en place demeurent limités et plusieurs améliorations peuvent être envisagées. Parmi elles figurent notamment la mise en œuvre de protections contre les attaques web, l'ajout d'une fonctionnalité de réinitialisation de mot de passe ainsi que l'intégration d'un système d'envoi automatique de courriels permettant d'informer les utilisateurs de l'évolution de leurs demandes.

2.2.4. Version 3 : Sécurité et communication

L'objectif de cette version est de renforcer la sécurité de l'application en mettant en place plusieurs mécanismes de protection, notamment le chiffrement des mots de passe ainsi que l'intégration d'un module de protection contre les attaques CSRF. Elle vise également à enrichir les fonctionnalités existantes en ajoutant l'envoi de courriels ainsi qu'un système de récupération et de modification de mot de passe.

Commençons par la partie sécurité. Comme évoqué lors de la version 1, celle-ci repose principalement sur un ensemble de critères que certaines données doivent respecter afin d'être validées. Par exemple, lors de l'inscription, le mot de passe doit répondre à plusieurs règles : contenir au minimum une majuscule, une minuscule, un chiffre ainsi qu'un caractère spécial issu d'une liste définie. De plus, sa longueur doit être d'au moins huit caractères.

D'autres contrôles sont également mis en place, notamment la validation des champs afin de garantir leur format, par exemple en s'assurant qu'un champ numérique ne contienne que des chiffres.

Cependant, ces vérifications sont principalement effectuées côté client, ce qui constitue une limite importante. En effet, ce type de contrôle peut facilement être contourné par un utilisateur mal intentionné, ce qui rend le système de sécurité insuffisant sans vérification côté serveur.

Une nouvelle stratégie est alors mise en place afin de renforcer cette protection. Dans un premier temps, chaque donnée est convertie dans son format attendu après validation, notamment pour les valeurs numériques afin d'éviter toute modification de type ou d'interprétation incorrecte.

Des contrôles supplémentaires sont également appliqués, avec des vérifications de longueur minimale et maximale, permettant de s'assurer que les données respectent un format cohérent et sécurisé.

Le format des adresses e-mail est également validé afin de garantir qu'il s'agit bien d'une adresse conforme et non d'une chaîne de caractères arbitraires. Les valeurs issues de listes prédéfinies sont systématiquement contrôlées afin d'empêcher l'injection de données non autorisées.

Dans le cas de la création d'un ticket, une vérification est effectuée pour s'assurer qu'un fichier a bien été déposé. Enfin, pour les champs textuels tels que le nom de fichier ou l'identité de l'utilisateur, les caractères potentiellement dangereux sont filtrés afin de limiter les risques d'injection de commandes ou de scripts.

Une fois la sécurité renforcée au niveau des entrées, une nouvelle étape consiste à protéger les données en cas de compromission de la base de données. En effet, si celle-ci venait à être exposée, il est essentiel de limiter les risques liés à l'utilisation frauduleuse des comptes utilisateurs.

Pour cela, les informations sensibles, et en particulier les mots de passe, ne sont pas stockées en clair dans la base de données. Elles sont transformées à l'aide d'un algorithme de hachage afin de rendre leur lecture impossible en cas de fuite. Contrairement au chiffrement, le hachage est un processus non réversible.

Dans ce projet, la bibliothèque bcrypt a été utilisée. Elle repose sur l'algorithme de Blowfish et permet de générer des empreintes de mots de passe robustes. Elle intègre également un facteur de coût (nombre d'itérations), ce qui permet d'augmenter volontairement le temps de calcul afin de renforcer la sécurité face aux attaques par force brute.

Une fois le niveau de sécurité renforcé, il est nécessaire de se protéger contre certaines attaques web, notamment les attaques de type CSRF (Cross-Site Request Forgery). Ce type d'attaque consiste à forcer un utilisateur authentifié à effectuer des actions non souhaitées au sein d'une application web dans laquelle il est déjà connecté.

Afin de prévenir ce risque, le middleware csurf a été utilisé. Il s'agit d'un module Node.js qui génère un jeton unique (token CSRF) associé à la session de l'utilisateur. Ce jeton est ensuite vérifié lors de chaque requête sensible. Si le token envoyé par le client ne correspond pas à celui attendu par le serveur, la requête est rejetée, empêchant ainsi toute action non autorisée.

Maintenant que le niveau de sécurité a été significativement renforcé, de nouvelles fonctionnalités ont pu être ajoutées à l'application. Dans cette version, deux fonctionnalités principales ont été mises en place.

La première concerne l'envoi de courriels. L'objectif est d'envoyer automatiquement un message à l'utilisateur lors de son inscription afin de confirmer la création de son compte. Bien que cette fonctionnalité soit encore limitée dans sa version actuelle, elle pourra être étendue ultérieurement en fonction des évolutions de l'application.

Pour sa mise en œuvre, le service Mailjet a été utilisé. Il s'agit d'une plateforme permettant l'envoi et le suivi d'e-mails via une API. Ce service propose également une documentation détaillée ainsi que des exemples de code facilitant la prise en main. Cela a permis de

simplifier l'intégration dans l'application et d'adapter rapidement les exemples fournis aux besoins du projet.

La seconde fonctionnalité concerne la gestion de l'oubli de mot de passe. L'objectif est de permettre à un utilisateur ayant perdu ses identifiants de réinitialiser son accès à son compte.

Pour cela, un bouton a été ajouté sur la page de connexion. Celui-ci ouvre un formulaire dans lequel l'utilisateur saisit son adresse e-mail. Après validation, un mot de passe temporaire est généré de manière aléatoire et envoyé à l'adresse renseignée. Ce mot de passe temporaire est composé de caractères variés afin de garantir un niveau de sécurité suffisant et d'éviter toute tentative d'accès non autorisé.

En complément, une fonctionnalité de modification du mot de passe a été mise en place. Une nouvelle page de profil a été créée, regroupant les informations de l'utilisateur telles que le nom, le prénom, l'adresse e-mail, le département et le rôle.

Fig 12 : Page Profil

Depuis cette page, l'utilisateur peut accéder à un formulaire de changement de mot de passe. Celui-ci demande l'ancien mot de passe ainsi que la saisie du nouveau mot de passe à deux reprises afin de limiter les erreurs de saisie. Lors de la validation, l'ancien mot de passe est vérifié avant d'être remplacé par le nouveau si la vérification est conforme.

Fig 13 : Page Profil - Changement Mot de passe

Cette version permet de conclure le développement du projet. L'application intègre désormais l'ensemble des fonctionnalités définies dans le cahier des charges, les différents rôles nécessaires à son utilisation ainsi que plusieurs mécanismes destinés à renforcer sa sécurité.

Bien que le niveau de sécurité ait été considérablement amélioré grâce aux différentes protections mises en place, aucune application ne peut être considérée comme totalement invulnérable. En effet, le risque zéro n'existe pas en informatique et la sécurité doit être envisagée comme un processus d'amélioration continue.

L'application est désormais suffisamment aboutie pour envisager son déploiement et son utilisation dans le cadre du service de reprographie, tout en conservant la possibilité d'intégrer de futures évolutions et améliorations.

2.2.5. Installation système

Maintenant que le développement de l'application est achevé, il est possible de passer au second objectif du projet : son déploiement. En effet, l'application a été conçue pour être accessible sur le réseau interne de l'établissement, tout en offrant la possibilité d'un accès à distance depuis un poste extérieur.

Afin de tester son déploiement, une Raspberry Pi m'a été mise à disposition. Celle-ci avait pour rôle d'héberger l'application web ainsi que la base de données MongoDB nécessaire à son fonctionnement. L'objectif était ensuite de configurer cet équipement afin qu'il soit accessible depuis les différents appareils connectés au réseau.

Pour faciliter cette phase, la Raspberry Pi contenait déjà un projet réalisé par un groupe d'étudiants de deuxième année. Ce projet utilisait également MongoDB et disposait déjà d'une configuration réseau opérationnelle. Cette base de travail m'a permis de me concentrer principalement sur l'installation, la configuration et la mise en service de l'application développée durant le stage.

Pour déployer l'application sur la Raspberry Pi, j'ai utilisé Git sous Ubuntu afin de récupérer directement le code source hébergé sur GitHub.

La première étape a consisté à installer Git à l'aide du gestionnaire de paquets du système. Une fois l'installation terminée, j'ai configuré mon identité Git en renseignant mon adresse électronique et mon nom d'utilisateur. Cette configuration permet notamment d'identifier les opérations effectuées sur les dépôts distants.

Après cette phase de configuration, j'ai pu cloner le dépôt contenant l'application sur la Raspberry Pi. Cette méthode présente l'avantage de récupérer l'intégralité du projet ainsi que son historique de versions, facilitant ainsi les mises à jour et la maintenance de l'application lors des différentes phases de déploiement.

Concernant la base de données, MongoDB étant déjà installé sur la Raspberry Pi, il m'a suffi de créer une nouvelle base de données ainsi que les collections nécessaires au fonctionnement de l'application.

Une fois l'application déployée sur la Raspberry Pi, il a été nécessaire de tester les connexions. Pour cela, la Raspberry a été connectée au réseau, puis l'accès à l'application a été vérifié depuis un autre appareil afin de s'assurer de sa disponibilité. Les différentes fonctionnalités ont également été testées pour chaque rôle. Ces tests ayant été concluants, le projet a été validé comme fonctionnel et pouvait être envisagé pour un déploiement sur un serveur.

Conclusion : Bilan et Perspectives

Même si ce stage n'a pas eu la même durée qu'un stage de troisième année, il a été particulièrement intéressant car il m'a permis d'apprendre et d'approfondir plusieurs technologies et outils.

Dans un premier temps, j'ai pu améliorer mes compétences en Node.js, notamment dans la mise en place de routes, de pages et de middlewares, ainsi que dans le développement des différentes fonctionnalités de l'application. J'ai également découvert et utilisé plusieurs modules de l'écosystème Node.js. Express m'a permis de disposer d'une base solide pour structurer l'application, express-fileupload m'a permis d'implémenter des fonctionnalités d'envoi et de téléchargement de fichiers, et Mongoose m'a facilité l'utilisation de MongoDB sans avoir à écrire de requêtes complexes, contrairement à un précédent projet réalisé avec PHP et MySQL.

Dans un second temps, j'ai pu découvrir un nouvel outil : Mailjet, une API permettant l'envoi automatique de courriels depuis une application web. L'utilisation de cette API a été source de difficultés, notamment lors de la création de mon compte, qui était bloqué car effectué depuis un réseau public (réseau universitaire). J'ai donc dû recréer un compte depuis un réseau personnel afin de pouvoir l'utiliser correctement. Malgré ces difficultés, la documentation et les tutoriels proposés par Mailjet ont facilité son intégration dans le projet.

Enfin, j'ai pu approfondir mes connaissances en matière de sécurité informatique. Je connaissais déjà l'importance du chiffrement des mots de passe et des données sensibles afin de protéger les utilisateurs. En revanche, j'ai découvert des attaques plus spécifiques comme le CSRF (Cross-Site Request Forgery). J'ai ainsi mis en place une protection grâce au module csurf, qui utilise des tokens afin de sécuriser les requêtes et de limiter ce type d'attaque.

Même si ce stage ne m'a pas permis de travailler en équipe, il m'a donné l'occasion de mettre en pratique l'ensemble des connaissances acquises au cours de mes années universitaires, puisque j'ai dû prendre en charge toutes les étapes du projet, de la conception jusqu'au déploiement.

De plus, le projet ayant évolué au fil du stage, cette charge de travail supplémentaire ne m'a pas posé de difficulté particulière et m'a permis d'aboutir à un résultat plus abouti que celui initialement attendu. Elle m'a également permis de développer des compétences en systèmes, bien que cet aspect n'ait pas été prévu au départ, mais qu'il m'a été confié en raison de la bonne avancée du projet.

Glossaire

Ticket : En informatique, un ticket désigne une demande enregistrée dans un système de gestion afin d'assurer son suivi. Il peut s'agir d'une demande d'assistance, d'un signalement d'incident ou d'une requête utilisateur. Dans le cadre de ce projet, un ticket correspond à une demande de reprographie soumise par un utilisateur et traitée par le service de reprographie.

Reprographie : Ensemble des techniques permettant la reproduction et l'impression de documents sur différents supports. Dans ce projet, le service de reprographie est chargé de traiter les demandes d'impression des utilisateurs.

Reprographe : Utilisateur chargé du traitement des demandes de reprographie. Il consulte les tickets, télécharge les documents, réalise les impressions et met à jour l'état des demandes.

Session utilisateur : Ensemble des informations conservées temporairement par le serveur afin d'identifier un utilisateur connecté pendant sa navigation sur l'application.

Document : Dans MongoDB, un document est l'unité de stockage des données. Il correspond à un ensemble d'informations organisées sous forme de paires champ-valeur (*field: value*), similaire à un objet JSON. Les documents sont stockés au format BSON (*Binary JSON*), une représentation binaire du format JSON permettant d'optimiser le stockage et les performances.

Express.js : Framework Node.js permettant de créer des applications web et des API.

Node.js : Environnement d'exécution JavaScript côté serveur.

Middleware : Fonction exécutée entre la requête et la réponse dans une application web.

Session : Mécanisme permettant de conserver les informations d'un utilisateur connecté.

API : Interface permettant la communication entre différentes parties d'une application.

Route : Chemin d'accès définissant une action dans une application web (ex : /login).

CSRF (Cross-Site Request Forgery) : Attaque forçant un utilisateur connecté à effectuer une action non voulue.

.

Sitographie /Bibliographie

Wikipédia Institut universitaire de technologie de Vélizy-Rambouillet
https://fr.wikipedia.org/wiki/Institut_universitaire_de_technologie_de_V%C3%A9lizy-Rambouillet

Wikipédia Université de Versailles – Saint-Quentin-en-Yvelines
https://fr.wikipedia.org/wiki/Universit%C3%A9_de_Versailles_%E2%80%93_Saint-Quentin-en-Yvelines

Wikipédia Express.js
https://fr.wikipedia.org/wiki/Express.js

Wikipédia Bcrypt
https://fr.wikipedia.org/wiki/Bcrypt